Chapter - 5

Advanced Python Programming (Turtle)

Turtle Graphics

- Turtle graphics is a simple but powerful kind of graphics programming.
- It has a turtle and a turtle window.
- The turtle moves around the turtle window and draws wherever it goes.
- You write *turtle commands* in the code to control the turtle.

Getting Started with the Turtle

- To start using the turtle in Python, you need to tell Python that you want to use the turtle commands, by typing this:
  - `import turtle`
- After that, you can start to draw things by using various turtle commands.
- When you finish drawing, you then use the `turtle.done` command to tell Python that you are done with the turtle.

A Turtle Graphics Program

- The basic structure of a turtle graphics program then looks like this:

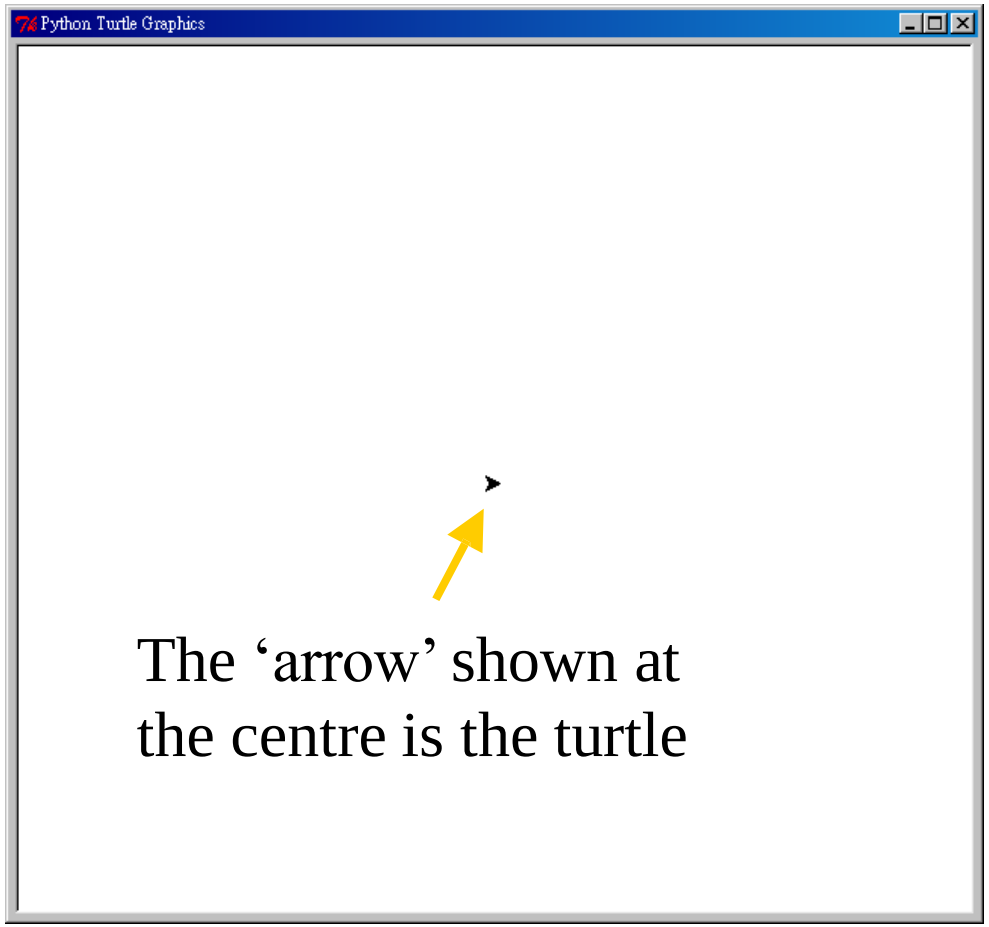
```
import turtle

. . .  Draw things using the turtle  . . .

turtle.done()
```

- You don't need to explicitly 'open' the turtle window; it is shown automatically the first time any turtle command is used in the program

The Turtle

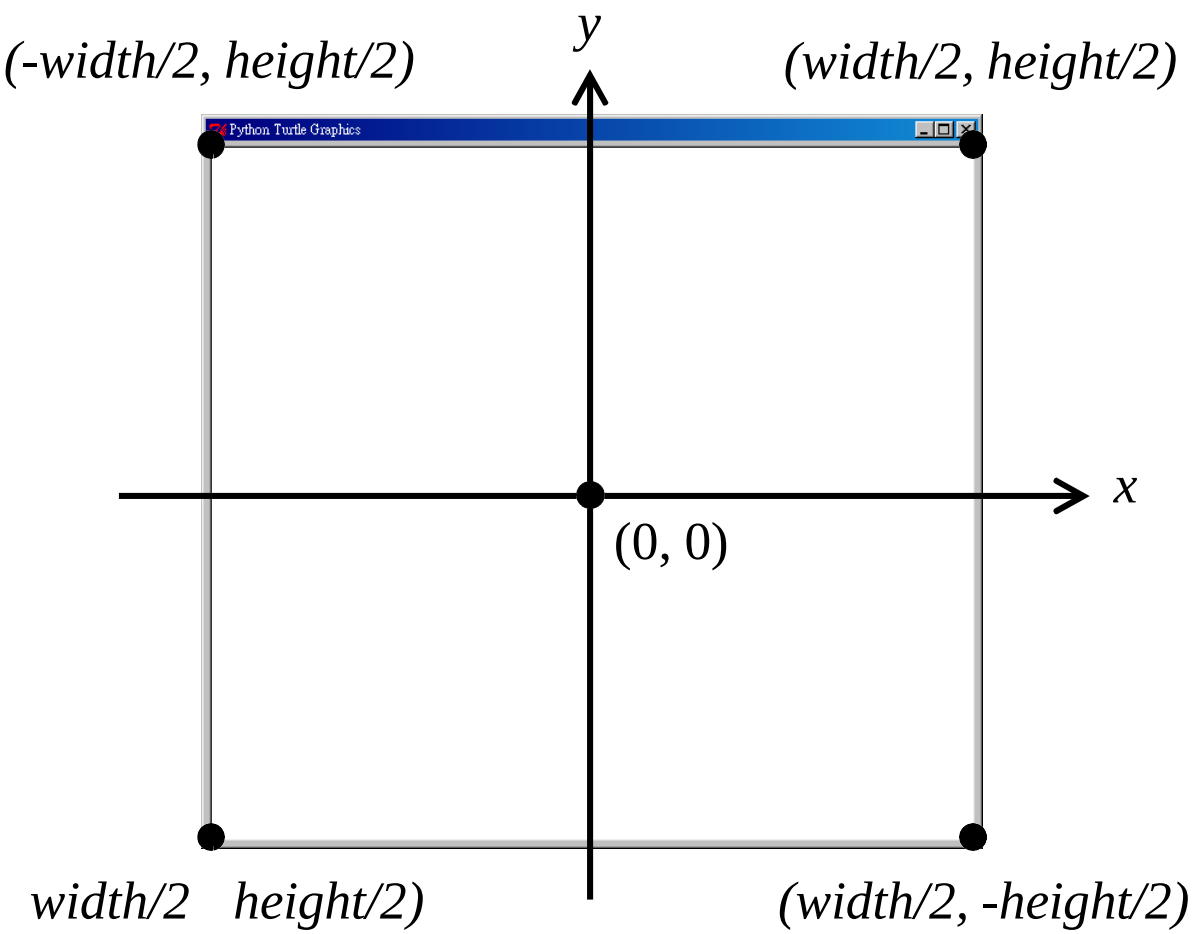
- The turtle has a position, an orientation and a pen
  - The position is where the turtle is inside the turtle window
  - The orientation is the direction the turtle is looking at, which is measured by an angle
  - The turtle holds a pen, which has a certain colour and thickness
- The turtle can put the pen down or lift the pen up.
- When the pen is down and the turtle moves, the turtle draws wherever it goes.

The Turtle Window



- The turtle moves around and draws inside the turtle window.
- Initially, the turtle starts at position (0, 0), which is in the middle of the window, and it looks towards the right hand side of the window

The Turtle Graphics Coordinate System



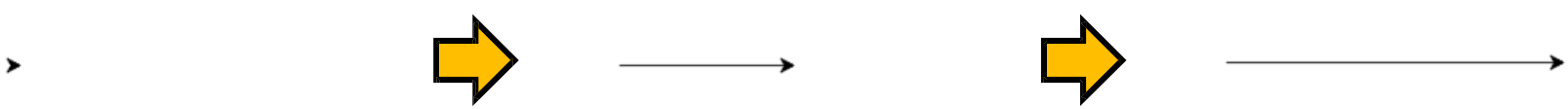
- You locate the turtle based on the turtle graphics coordinate system.
- The basic unit of the system is called a *pixel*, which is the display unit on a computer screen.

Walking Forwards

- You can ask the turtle to walk forwards.
- For example, to tell the turtle to walk 250 pixels forwards, you can use this line of code:

```
turtle.forward(250)
```

- After running the code, you can actually see the turtle moving from one position to another



- When the turtle moves, it draws a line

Walking Backwards

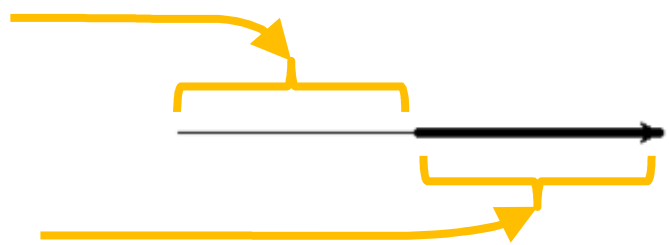
- Similarly, you can ask the turtle to move backwards using the `turtle.backward` command
- For example, the following line of code asks the turtle to move backwards by 200 pixels:

```
turtle.backward(200)
```

Changing the Line Thickness

- You can draw a thicker line by changing the *width* of the pen the turtle is holding using the `turtle.width` command
- The following code draws a thin line and then a thick line by changing the width of the pen to 5 pixels:

```
turtle.forward(125)
turtle.width(5)
turtle.forward(125)
```



Turning Left and Right

- The turtle can do much more than walking forwards and backwards horizontally
- You can turn turtle using the `turtle.left` and `turtle.right` commands
- For example, to turn left by 90 degrees:

```
turtle.left(90)
```

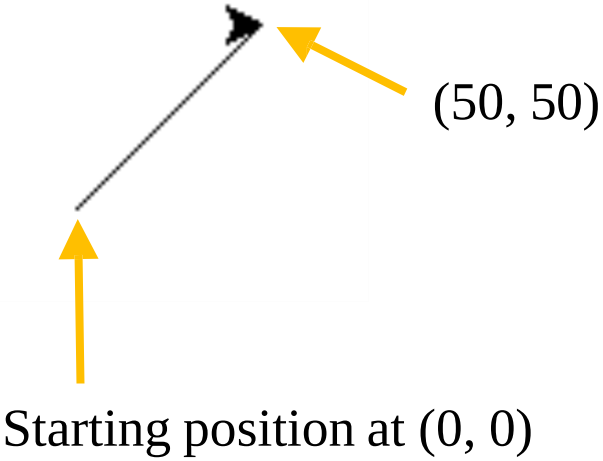
- And, to turn right by 45 degrees:

```
turtle.right(45)
```

Moving to an Arbitrary Location

- The turtle can move around the window by walking forwards, backwards and turning.
- Sometimes it may be useful to ask the turtle to move directly to a particular position in the window.
- To do that you can use the `turtle.goto` command
- For example, you can ask the turtle to move to the location (50, 50) using this line of code:

```
turtle.goto(50, 50)
```



Moving Back Home

- The initial position of the turtle, i.e. (0, 0), is called the home of the turtle
- You can quickly move the turtle back to its home using this line of code:

```
turtle.home()
```

- This is the same as running this code:

```
turtle.goto(0, 0)
```

Lifting Up and Putting Down the Pen

- By default, when the turtle moves, the pen draws
- If you want the turtle to move without drawing, you tell the turtle to lift up the pen:

```
turtle.up()
```

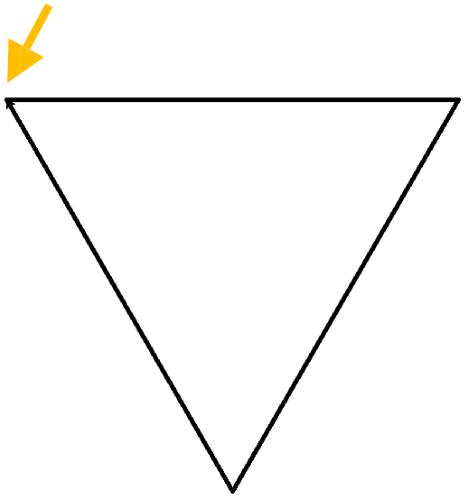
- Nothing is drawn when the turtle moves after that
- To let the turtle draw again, tell the turtle to put the pen down using this code:

```
turtle.down()
```

Drawing Shapes

- We can draw lots of different shapes with lines
- Here is an example that draws a triangle

```
turtle.forward(500)
turtle.right(120)
turtle.forward(500)
turtle.right(120)
turtle.forward(500)
```



- A hollow triangle is drawn

Drawing Shapes

- Here is an example that draws a rectangle

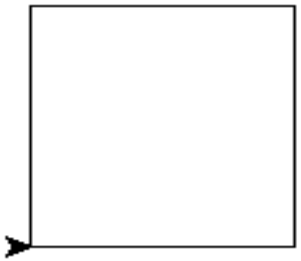
```
turtle.forward(300)
turtle.right(90)
turtle.forward(150)
turtle.right(90)
turtle.forward(300)
turtle.right(90)
turtle.forward(150)
turtle.right(90)
```

- A hollow rectangle is drawn.

Drawing Shapes

- Here is an example that draws a square

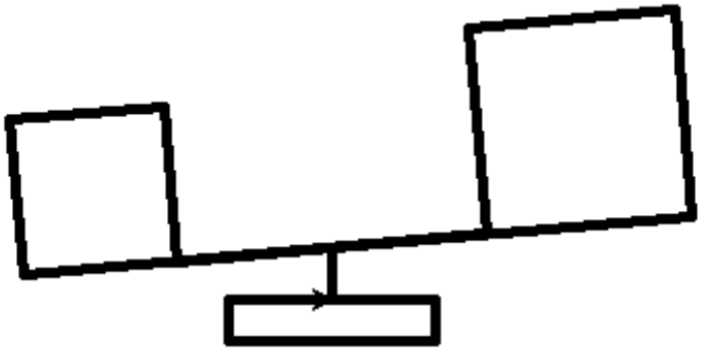
```
turtle.forward(200)
turtle.left(90)
turtle.forward(200)
turtle.left(90)
turtle.forward(200)
turtle.left(90)
turtle.forward(200)
turtle.left(90)
```



- A hollow square is drawn.

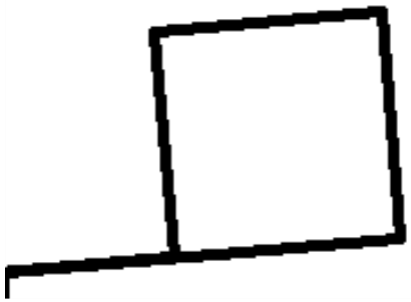
An Example Using Multiple Commands

- This example demonstrates how the turtle reacts to a combination of the turtle commands that we have met so far
- Result:



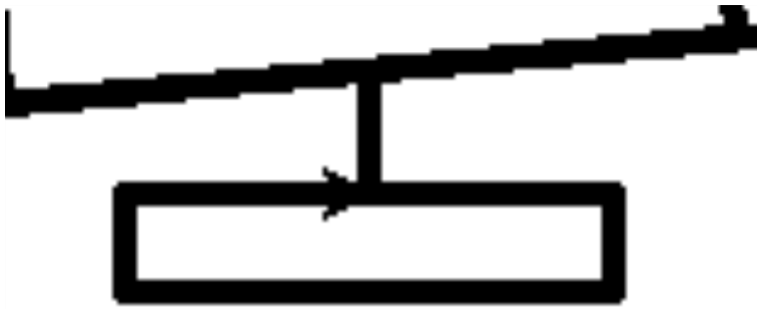
Example Code 1/2

```
turtle.left(5) # Tell the turtle to turn left 5 degrees

# Draw the right box and the connecting line
turtle.forward(175)
turtle.left(90)
turtle.forward(100)
turtle.left(90)
turtle.forward(100)
turtle.left(90)
turtle.forward(100)

turtle.right(90)

# Draw the left box and the connecting line
turtle.forward(225)
turtle.right(90)
turtle.forward(75)
turtle.right(90)
turtle.forward(75)
turtle.right(90)
turtle.forward(75)
```



Example Code 2/2

```
# Return to the centre
turtle.left(90)
turtle.forward(75)
turtle.right(95)

# Draw the base
turtle.forward(25)
turtle.left(90)
turtle.forward(50)
turtle.right(90)
turtle.forward(20)
turtle.right(90)
turtle.forward(100)
turtle.right(90)
turtle.forward(20)
turtle.right(90)
turtle.forward(50)
```



Drawing Solid Shapes

- You can tell the turtle to draw filled shapes
- Before drawing a shape, you need to run this line of code:

```
turtle.begin_fill()
```

- And after finishing the shape, use this line of code:

```
turtle.end_fill()
```

- The shape will then be immediately filled with the *fill colour*

Changing the Pen Colour

- You can easily change the colours
- The initial colour of the pen is black
- You can change the pen colour (=line colour) using the `turtle.color` command
- Here is an example changing the pen colour to red:

```
turtle.color("red")
```

- And to change the pen colour to yellow:

```
turtle.color("yellow")
```

Changing the Fill Colour

- You can also change the fill colour using the `turtle.fillcolor` command, like this:

```
turtle.fillcolor("yellow")
```

- If you want to, you can change both colours at the same time:

Line colour Fill colour

```
turtle.color("red", "yellow")
```

- After this line, every line drawn by the turtle will be red and every filled shape will be filled with yellow

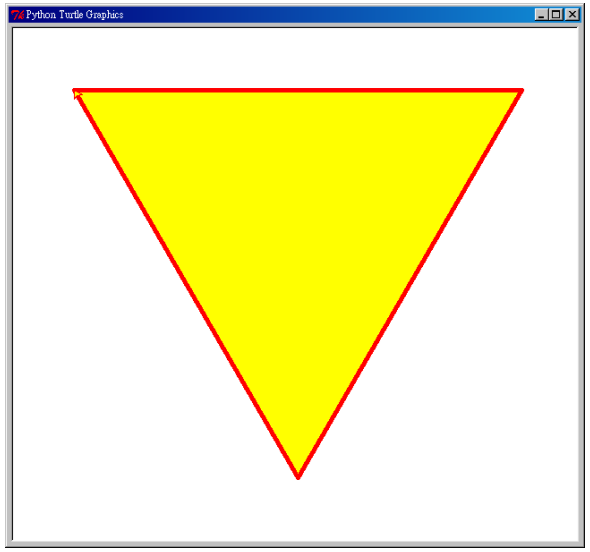
An Example of Using Colours

- This example illustrates how to draw a yellow triangle with a red border

```
turtle.color("red", "yellow")

turtle.begin_fill()

turtle.forward(500)
turtle.right(120)
turtle.forward(500)
turtle.right(120)
turtle.forward(500)

turtle.end_fill()
```

Draw the triangle

Fill the shape drawn between these two lines

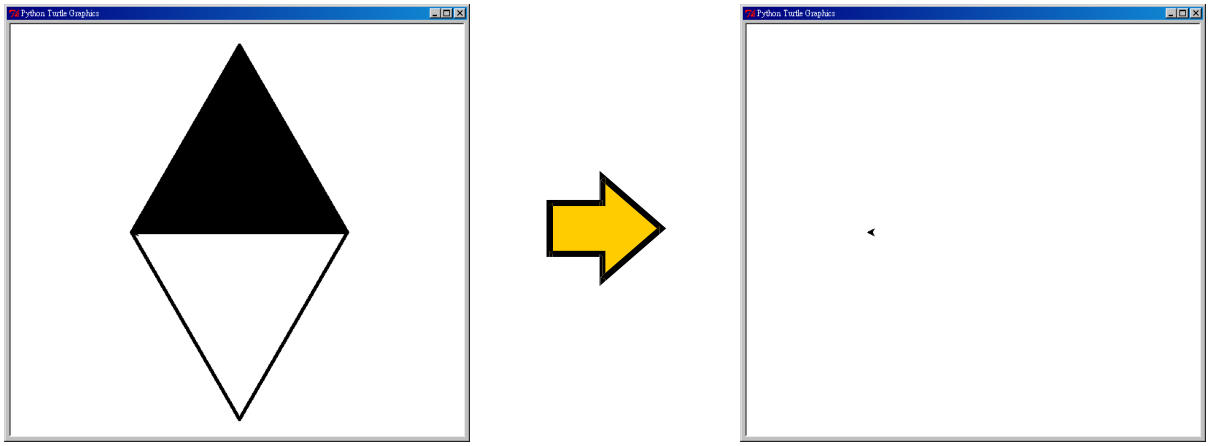


Clearing the Screen

- To clear everything drawn so far, run the following line of code:

```
turtle.clear()
```

- The position and orientation of the turtle remains the same as whatever it was before running the code



Resetting Everything

- If you want to clear the turtle window as well as resetting everything to the initial state, you can use `turtle.reset()`
- After running the command, everything such as the turtle position, orientation and colours will be back to their initial values
- For example, the turtle will move back home and look to the right hand side of the window

Speeding Up the Turtle

- The turtle initially moves fairly slowly
- You can adjust the moving speed of the turtle using the `turtle.speed` command
- The initial speed value is 3
- For example, you can make the turtle move faster by running this line of code:

```
turtle.speed(8)
```

The Turtle Speed Value

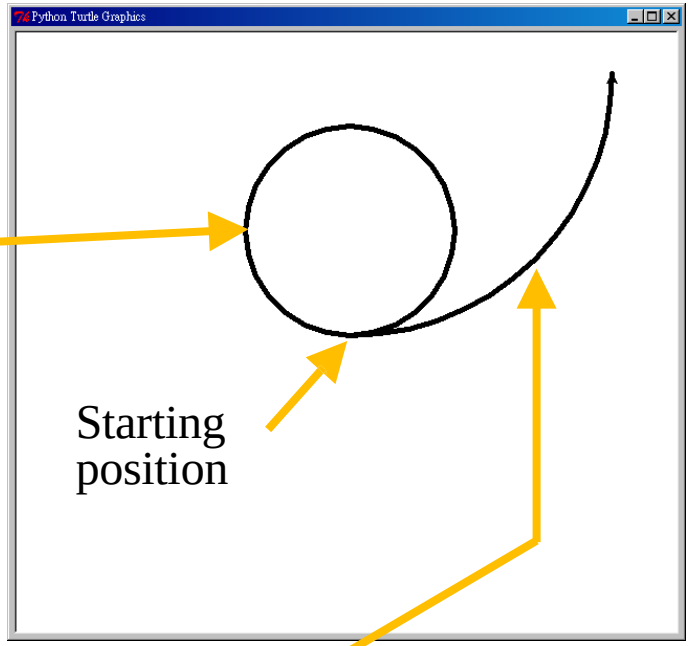
- The speed value is an integer from 0 to 10.
- The speed changes from very slow to very fast from 1 to 10, i.e.
  - 1=slow speed … 6=normal speed … 10=fast speed
- A speed value of 0 means no animation at all.
- If you use the value of 0 you will still see the turtle ‘jump’ immediately from one position to another instead of moving gradually.

Drawing Circles and Arcs

- To draw a circle, use `circle()`:

```
turtle.circle( 100 )
```

radius=100 pixels



- You can also draw part of a circle:

```
turtle.circle( 250 , 90 )
```

The centre of the circle is 250 pixels left of the turtle

Draw 90 degrees of that circle

Hiding the Turtle

- After the turtle has finished drawing, the turtle is still shown
- It doesn't look good on top of the teddy bear image we have drawn
- We can make the turtle disappear:

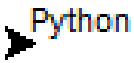
```
turtle.hideturtle()
```

Writing Text Using the Turtle

- You can display text using the turtle
- For example, you can use the code below to write the text ‘COMP1021’ in the turtle window:

```
turtle.write("Python")
```

- The text is shown all at once instead of being gradually drawn



Customising the Font

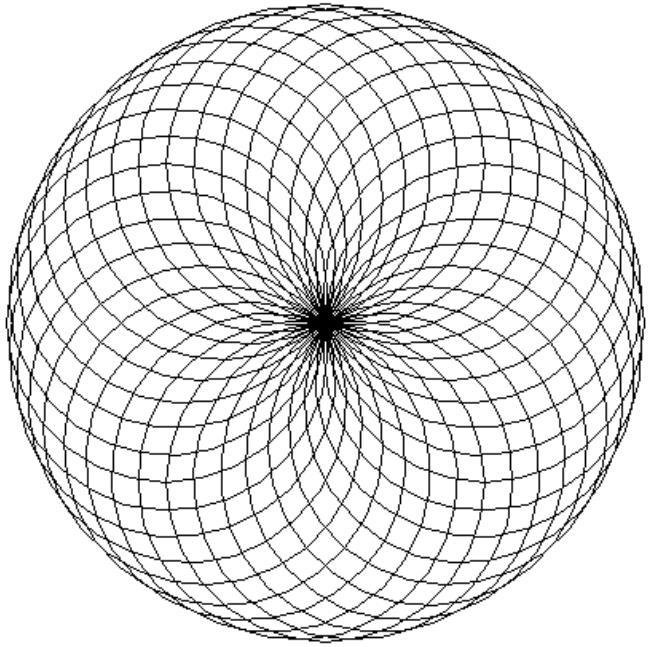
- If you want to, you can customise the font using the font option
- For example, a bigger and bold font can be used to write the text ‘COMP1021’ in the window

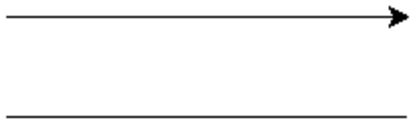
```
turtle.write("Python",
    font=("Arial", 20, "bold"))
```

Using the bold ‘Arial’ font with a size of 20

Flower Example

```
import turtle
t = turtle.Turtle()
t.speed(0)
for i in range(36):
    t.circle(100)
    t.left(10)

turtle.done()
```



Drawing Two Separate Lines Example

```
import turtle

t = turtle.Turtle()
t.speed(1)
# First line
t.forward(200)
t.penup()
t.goto(0,50)
# Draw second line
t.pendown()
t.forward(200)
turtle.done()
```